```
import tensorflow as tf
import numpy as np
from tensorflow.keras import layers, models
from tensorflow.keras.applications.resnet50 import preprocess_input# Replace 'your_filename.zip' with the actual name you
!unzip -o archive.zip
```

```
inflating: NEU-DET/validation/images/scratches/scratches_243.jpg
inflating: NEU-DET/validation/images/scratches/scratches_244.jpg
inflating: NEU-DET/validation/images/scratches/scratches_245.jpg
inflating: NEU-DET/validation/images/scratches/scratches_246.jpg
inflating: NEU-DET/validation/images/scratches/scratches_247.jpg
inflating: NEU-DET/validation/images/scratches/scratches_248.jpg
inflating: NEU-DET/validation/images/scratches/scratches_249.jpg
inflating: NEU-DET/validation/images/scratches/scratches_250.jpg
inflating: NEU-DET/validation/images/scratches/scratches_251.jpg
inflating: NEU-DET/validation/images/scratches/scratches_252.jpg
inflating: NEU-DET/validation/images/scratches/scratches_253.jpg
inflating: NEU-DET/validation/images/scratches/scratches_254.jpg
inflating: NEU-DET/validation/images/scratches/scratches_255.jpg
inflating: NEU-DET/validation/images/scratches/scratches_256.jpg
inflating: NEU-DET/validation/images/scratches/scratches_257.jpg
inflating: NEU-DET/validation/images/scratches/scratches_258.jpg
inflating: NEU-DET/validation/images/scratches/scratches_259.jpg
inflating: NEU-DET/validation/images/scratches/scratches_260.jpg
inflating: NEU-DET/validation/images/scratches/scratches_261.jpg
inflating: NEU-DET/validation/images/scratches/scratches_262.jpg
inflating: NEU-DET/validation/images/scratches/scratches_263.jpg
inflating: NEU-DET/validation/images/scratches/scratches_264.jpg
inflating: NEU-DET/validation/images/scratches/scratches_265.jpg
inflating: NEU-DET/validation/images/scratches/scratches_266.jpg
inflating: NEU-DET/validation/images/scratches/scratches_267.jpg
inflating: NEU-DET/validation/images/scratches/scratches_268.jpg
inflating: NEU-DET/validation/images/scratches/scratches_269.jpg
inflating: NEU-DET/validation/images/scratches/scratches_270.jpg
inflating: NEU-DET/validation/images/scratches/scratches_271.jpg
inflating: NEU-DET/validation/images/scratches/scratches_272.jpg
inflating: NEU-DET/validation/images/scratches/scratches_273.jpg
inflating: NEU-DET/validation/images/scratches/scratches_274.jpg
inflating: NEU-DET/validation/images/scratches/scratches_275.jpg
inflating: NEU-DET/validation/images/scratches/scratches_276.jpg
inflating: NEU-DET/validation/images/scratches/scratches_277.jpg
inflating: NEU-DET/validation/images/scratches/scratches_278.jpg
inflating: NEU-DET/validation/images/scratches/scratches_279.jpg
inflating: NEU-DET/validation/images/scratches/scratches_280.jpg
inflating: NEU-DET/validation/images/scratches/scratches_281.jpg
inflating: NEU-DET/validation/images/scratches/scratches_282.jpg
inflating: NEU-DET/validation/images/scratches/scratches_283.jpg
inflating: NEU-DET/validation/images/scratches/scratches_284.jpg
inflating: NEU-DET/validation/images/scratches/scratches_285.jpg
inflating: NEU-DET/validation/images/scratches/scratches_286.jpg
inflating: NEU-DET/validation/images/scratches/scratches_287.jpg
inflating: NEU-DET/validation/images/scratches/scratches_288.jpg
inflating: NEU-DET/validation/images/scratches/scratches_289.jpg
inflating: NEU-DET/validation/images/scratches/scratches_290.jpg
inflating: NEU-DET/validation/images/scratches/scratches_291.jpg
inflating: NEU-DET/validation/images/scratches/scratches_292.jpg
inflating: NEU-DET/validation/images/scratches/scratches_293.jpg
inflating: NEU-DET/validation/images/scratches/scratches_294.jpg
inflating: NEU-DET/validation/images/scratches/scratches_295.jpg
inflating: NEU-DET/validation/images/scratches/scratches_296.jpg
inflating: NEU-DET/validation/images/scratches/scratches_297.jpg
inflating: NEU-DET/validation/images/scratches/scratches_298.jpg
inflating: NEU-DET/validation/images/scratches/scratches_299.jpg
inflating: NEU-DET/validation/images/scratches/scratches_300.jpg
```

```
# The main folder that was extracted
base_dir = 'NEU-DET'

from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Setup training and validation generators
datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    horizontal_flip=True
)

# NEU-DET is already split into train/validation folders in your zip
```

```
train_generator = datagen.flow_from_directory(
    'NEU-DET/train/images', # Point to the train images
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)

val_generator = datagen.flow_from_directory(
    'NEU-DET/validation/images', # Point to the validation images
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)
```

```
Found 1440 images belonging to 6 classes.
Found 360 images belonging to 6 classes.
```

```
from tensorflow.keras.applications import ResNet50
from tensorflow.keras import layers, models

# Load ResNet50
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False

# Custom Head
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dense(train_generator.num_classes, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

```
history = model.fit(train_generator, validation_data=val_generator, epochs=10)
```

```
Epoch 1/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 48s 740ms/step - accuracy: 0.1688 - loss: 1.8429 - val_accuracy: 0.1667 - val_loss: 1.7911
Epoch 2/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 24s 545ms/step - accuracy: 0.2111 - loss: 1.7813 - val_accuracy: 0.1972 - val_loss: 1.8254
Epoch 3/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 23s 505ms/step - accuracy: 0.3069 - loss: 1.7316 - val_accuracy: 0.2472 - val_loss: 1.6924
Epoch 4/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 23s 510ms/step - accuracy: 0.2618 - loss: 1.6839 - val_accuracy: 0.3639 - val_loss: 1.6395
Epoch 5/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 23s 521ms/step - accuracy: 0.3431 - loss: 1.6248 - val_accuracy: 0.2583 - val_loss: 1.6049
Epoch 6/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 24s 530ms/step - accuracy: 0.3646 - loss: 1.5922 - val_accuracy: 0.2889 - val_loss: 1.6132
Epoch 7/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 23s 514ms/step - accuracy: 0.3306 - loss: 1.5859 - val_accuracy: 0.4194 - val_loss: 1.5656
Epoch 8/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 23s 504ms/step - accuracy: 0.4549 - loss: 1.4874 - val_accuracy: 0.3806 - val_loss: 1.4913
Epoch 9/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 22s 479ms/step - accuracy: 0.3979 - loss: 1.4686 - val_accuracy: 0.4417 - val_loss: 1.4699
Epoch 10/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 23s 507ms/step - accuracy: 0.4694 - loss: 1.4317 - val_accuracy: 0.4028 - val_loss: 1.4381
```

```
import numpy as np
from sklearn.metrics import classification_report

# Predict the classes for the validation set
Y_pred = model.predict(val_generator)
y_pred = np.argmax(Y_pred, axis=1)

# Get the true labels
y_true = val_generator.classes

# Print the report
print(classification_report(y_true, y_pred, target_names=val_generator.class_indices.keys()))
```

```
12/12 ━━━━━━━━━━━━━━━━━━━━ 12s 634ms/step
                precision    recall  f1-score   support

       crazing       0.19      0.45      0.27        60
     inclusion       0.18      0.20      0.19        60
       patches       0.33      0.07      0.11        60
```

```
 pitted_surface       0.19      0.42      0.26        60
rolled-in_scale       0.00      0.00      0.00        60
      scratches       0.22      0.03      0.06        60

       accuracy                           0.19       360
      macro avg       0.19      0.19      0.15       360
   weighted avg       0.19      0.19      0.15       360

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-def
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-def
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-def
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
def get_factory_decision(prediction_array):
    # Get the index of the highest probability
    class_idx = np.argmax(prediction_array)
    probability = np.max(prediction_array)

    # Logic: If confidence is high (> 85%), take action
    if probability >= 0.85:
        return f"ACTION: Trigger rejection for {list(val_generator.class_indices.keys())[class_idx]}"
    else:
        return "DECISION: Product passed (Low defect confidence)"

# Test with a single image from the validation set
# Use the Python built-in next() function
img, label = next(val_generator)

# Now run the prediction
sample_prediction = model.predict(img[0:1])
print(get_factory_decision(sample_prediction))
sample_prediction = model.predict(img[0:1])
print(get_factory_decision(sample_prediction))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 5s 5s/step
DECISION: Product passed (Low defect confidence)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step
DECISION: Product passed (Low defect confidence)
```

```
# 1. Unfreeze the base model
base_model.trainable = True

# 2. Freeze all layers except the last 20 (This is the "Fine-Tuning" strategy)
for layer in base_model.layers[:-20]:
    layer.trainable = False

# 3. IMPORTANT: Recompile with a VERY small learning rate
# High rates will destroy the pre-trained weights
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# 4. Retrain for 5-10 more epochs
history_fine = model.fit(train_generator, validation_data=val_generator, epochs=5)
```

```
Epoch 1/5
45/45 ━━━━━━━━━━━━━━━━━━━━ 46s 648ms/step - accuracy: 0.4576 - loss: 2.8739 - val_accuracy: 0.2500 - val_loss: 1.7249
Epoch 2/5
45/45 ━━━━━━━━━━━━━━━━━━━━ 22s 490ms/step - accuracy: 0.7479 - loss: 0.7678 - val_accuracy: 0.2000 - val_loss: 2.1929
Epoch 3/5
45/45 ━━━━━━━━━━━━━━━━━━━━ 23s 507ms/step - accuracy: 0.8076 - loss: 0.6320 - val_accuracy: 0.1667 - val_loss: 2.3217
Epoch 4/5
45/45 ━━━━━━━━━━━━━━━━━━━━ 23s 507ms/step - accuracy: 0.8208 - loss: 0.5755 - val_accuracy: 0.1861 - val_loss: 2.3628
Epoch 5/5
45/45 ━━━━━━━━━━━━━━━━━━━━ 23s 508ms/step - accuracy: 0.8458 - loss: 0.5025 - val_accuracy: 0.1861 - val_loss: 2.3649
```

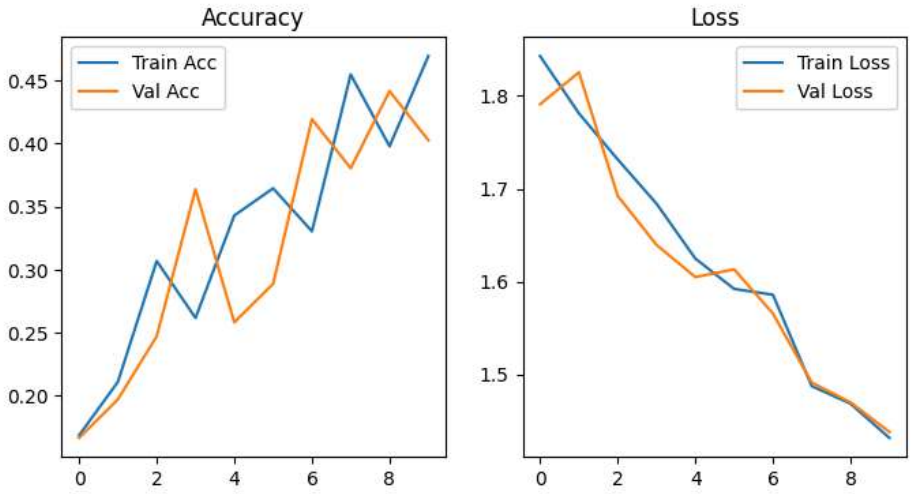
```
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),  # Added dropout to prevent overfitting
    layers.Dense(128, activation='relu'),
    layers.Dense(train_generator.num_classes, activation='softmax')
])
```

```
import matplotlib.pyplot as plt

# Assuming 'history' is the object from your first training phase
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
plt.plot(acc, label='Train Acc')
plt.plot(val_acc, label='Val Acc')
plt.legend()
plt.title('Accuracy')

plt.subplot(1, 2, 2)
plt.plot(loss, label='Train Loss')
plt.plot(val_loss, label='Val Loss')
plt.legend()
plt.title('Loss')
plt.show()
```




```
import os
# Point to the folder that contains your category folders
# Example: If your images are in 'NEU-DET/train/images'
data_dir = 'NEU-DET/train/images'

from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

train_generator = datagen.flow_from_directory(
    data_dir,
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)

print("Data loaded successfully!")
```

```
Found 1440 images belonging to 6 classes.
Data loaded successfully!
```

```
Start coding or generate with AI.
```